Xinyi Cui
xc535
Turned in A2 on April 25, using 1 split day

```
  Tie_Breaking   First Second   Third  Fourth
1       Single 16.0676 16.835 16.8576 10.2398
2     Multiple 16.0280 16.851 16.9096 10.2354
>
```

For Single: 16.0676 + 16.835 + 16.8576 + 10.2398 ≈ 60; for Multiple: 16.0280 + 16.851 + 16.9096 + 10.2354 ≈ 60. This confirms all 60 students are assigned in each simulation.

a) **Single Tie-Breaking:** A single tie-breaking rule offers significant practical advantages by streamlining the school assignment process. It requires only one centralized lottery to break ties across all schools, reducing administrative effort, time, and resources compared to managing multiple lotteries. This simplicity enhances efficiency, as the district can allocate resources more effectively. From a market design perspective, single tie-breaking maintains stability and strategyproofness within the deferred acceptance algorithm: stability is guaranteed as the algorithm prevents blocking pairs, and strategyproofness ensures students have no incentive to misreport preferences, as the random lottery is exogenous to their submissions. **Multiple Tie-Breaking:** A multiple tie-breaking rule provides practical advantages centered on fairness and equity, though at the cost of increased complexity. By using a separate lottery for each school, it gives students independent chances to rank higher at each school, reducing the impact of a single poor lotter. However, this approach reduces operational efficiency due to the need for more lotteries, increasing administrative burden. Despite these drawbacks, multiple tie-breaking also maintains stability and strategyproofness within the deferred acceptance framework: stability is ensured by the algorithm's structure, and strategyproofness is preserved as the independent lotteries remain random and unaffected by preference reporting, giving students no incentive to misreport.

b) More recommend single tie-breaking. The simulations show that single tie-breaking assigns slightly more students to their first preference (16.0676 vs. 16.0280, a difference of 0.0396 students) compared to multiple tie-breaking, aligning with the district's primary goal of maximizing first-choice assignments. The difference in fourth-choice assignments is even smaller (0.0044 students). When outcomes are this close, practical considerations become critical. Both methods ensure stability (via the deferred acceptance algorithm, preventing blocking pairs) and strategyproofness (students have no incentive to misreport preferences, as lotteries are random), so they are equivalent in these aspects. However, single tie-breaking is more efficient to implement, requiring only one lottery compared to four for multiple tie-breaking. If fairness were prioritized over simplicity, multiple tie-breaking could be considered, but the district's goals favor single tie-breaking.

Next Page shows the code

```
install.packages("matchingMarkets")
library(matchingMarkets)

# Set seed for reproducibility
set.seed(42)

# Problem parameters
nStudents <- 60
nSchools <- 4
capacities <- c(5, 40, 15, 10)  # School capacities
weights <- c(5, 1, 2, 3)  # Preference weights
nSimulations <- 5000

# School priority matrix based on the given table
school_priorities <- matrix(NA, nrow = nStudents, ncol = nSchools)
colnames(school_priorities) <- paste0("S", 1:nSchools)
rownames(school_priorities) <- paste0("St", 1:nStudents)

# Assign priorities
for (i in 1:nStudents) {
  # School 1
  if (i %in% 1:15) school_priorities[i, 1] <- 1
  else if (i %in% 16:30) school_priorities[i, 1] <- 2
  else if (i %in% 31:45) school_priorities[i, 1] <- 3
  else school_priorities[i, 1] <- 4

  # School 2
  if (i %in% 46:60) school_priorities[i, 2] <- 1
  else if (i %in% 1:15) school_priorities[i, 2] <- 2
  else if (i %in% 16:30) school_priorities[i, 2] <- 3
  else school_priorities[i, 2] <- 4

  # School 3
  if (i %in% 31:45) school_priorities[i, 3] <- 1
  else if (i %in% 46:60) school_priorities[i, 3] <- 2
  else if (i %in% 1:15) school_priorities[i, 3] <- 3
  else school_priorities[i, 3] <- 4

  # School 4
  if (i %in% 16:30) school_priorities[i, 4] <- 1
  else if (i %in% 31:45) school_priorities[i, 4] <- 2
  else if (i %in% 46:60) school_priorities[i, 4] <- 3
  else school_priorities[i, 4] <- 4
}

# Function to generate student preferences with weights
```

```
generate_preferences <- function(nStudents, nSchools, weights) {
  s.prefs <- matrix(NA, nrow = nSchools, ncol = nStudents, dimnames = list(paste0("S",
1:nSchools), paste0("St", 1:nStudents)))
  for (i in 1:nStudents) {
    # Sample school names with weights
    prefs <- sample(paste0("S", 1:nSchools), size = nSchools, replace = FALSE, prob = weights)
    s.prefs[, i] <- prefs
  }
  return(s.prefs)
}

# Function to convert priorities to preference lists
priorities_to_prefs <- function(priorities, tie_breaking = "single") {
  nStudents <- nrow(priorities)
  nSchools <- ncol(priorities)
  c.prefs <- matrix(NA, nrow = nStudents, ncol = nSchools, dimnames = list(paste0("St",
1:nStudents), paste0("S", 1:nSchools)))

  if (tie_breaking == "single") {
    # Single tie-breaking: one random lottery
    lottery <- sample(1:nStudents)
    for (j in 1:nSchools) {
      # Sort students by priority, then lottery
      students <- order(priorities[, j], lottery)
      c.prefs[, j] <- paste0("St", students)
    }
  } else {
    # Multiple tie-breaking: separate lottery per school
    for (j in 1:nSchools) {
      lottery <- sample(1:nStudents)
      students <- order(priorities[, j], lottery)
      c.prefs[, j] <- paste0("St", students)
    }
  }
  return(c.prefs)
}

# Function to count preference ranks
count_preferences <- function(matchings, s.prefs) {
  counts <- c(first = 0, second = 0, third = 0, fourth = 0)
  for (i in 1:nrow(matchings)) {
    student <- matchings$student[i]
    school <- matchings$college[i]
    if (is.na(student) || is.na(school)) next
    # Find rank of assigned school in student's preference
    student_col <- which(colnames(s.prefs) == student)
```

```
    if (length(student_col) == 0) next
    rank <- which(s.prefs[, student_col] == school)
    if (length(rank) == 0) next
    if (rank == 1) counts["first"] <- counts["first"] + 1
    else if (rank == 2) counts["second"] <- counts["second"] + 1
    else if (rank == 3) counts["third"] <- counts["third"] + 1
    else counts["fourth"] <- counts["fourth"] + 1
  }
  return(counts)
}

# Initialize results storage
single_results <- matrix(0, nrow = nSimulations, ncol = 4)
colnames(single_results) <- c("first", "second", "third", "fourth")
multiple_results <- matrix(0, nrow = nSimulations, ncol = 4)
colnames(multiple_results) <- c("first", "second", "third", "fourth")

# Run simulations
for (sim in 1:nSimulations) {
  # Generate student preferences
  s.prefs <- generate_preferences(nStudents, nSchools, weights)

  # Single tie-breaking
  c.prefs_single <- priorities_to_prefs(school_priorities, "single")
  result_single <- hri(s.prefs = s.prefs, c.prefs = c.prefs_single, nSlots = capacities,
check_consistency = TRUE)
  matchings_single <- result_single$matchings
  single_results[sim, ] <- count_preferences(matchings_single, s.prefs)

  # Multiple tie-breaking
  c.prefs_multiple <- priorities_to_prefs(school_priorities, "multiple")
  result_multiple <- hri(s.prefs = s.prefs, c.prefs = c.prefs_multiple, nSlots = capacities,
check_consistency = TRUE)
  matchings_multiple <- result_multiple$matchings
  multiple_results[sim, ] <- count_preferences(matchings_multiple, s.prefs)
}

# Summarize results
single_means <- colMeans(single_results)
multiple_means <- colMeans(multiple_results)

# Create data frame for results
results_df <- data.frame(
  Tie_Breaking = c("Single", "Multiple"),
  First = c(single_means["first"], multiple_means["first"]),
  Second = c(single_means["second"], multiple_means["second"]),
```

```
  Third = c(single_means["third"], multiple_means["third"]),
  Fourth = c(single_means["fourth"], multiple_means["fourth"])
)
print(results_df)
```